

SCAVY

Automating the discovery of Linux kernel privilege escalation primitives

Erin Avllazagaj, Tudor Dumitraş
University of Maryland, College Park

Abstract

With the ever-changing Linux kernel code base, commonly used primitives in exploit development are always at the risk of becoming obsolete while new ones may appear. Usually an author with expertise in a subsystem shares their exploit primitive and the others reuse it in their exploits. Should this primitive become obsolete the exploit author needs to go back to reading the kernel source code about unrelated kernel subsystems to find new kernel objects to target for their kernel memory corruption primitives. It's not realistic to have the exploit author be an expert in every subsystem of the Linux kernel. Additionally, from the defenders' perspective defenses are deployed after a primitive is publicized.

In this paper we propose SCAVY, a framework that can systematically and automatically find useful exploitation targets that lead the kernel into exploitable states. Our framework reveals 17 new fields in 11 kernel structures that when corrupted enable the exploit author to achieve privilege escalation.

Moreover, SCAVY shows that it can find 3 previously known targets that can still be used for privilege escalation on Linux kernel 5.15. This framework aids the exploit author during the exploit development stage by suggesting him potential targets for his particular memory corruption capability that can reach an exploitable state.

1 Introduction

During the Linux kernel exploit development stage the attacker is usually searching for the kernel structure that when corrupted by a memory overwrite capability can reach his desired exploitable state. For a local attacker in most cases the ultimate exploitable state is a privilege escalation. Many Proof of Concept exploits and prior work that facilitate bug exploitation [3, 38] ultimately reach kernel-context code execution, also known as Instruction Pointer hijacking, at which point the system is considered exploited. However, at the time of writing there are more than 2432 structs defined in the Linux kernel version 5.15.80 where only 142(5%) have at least 1 function pointer and 604 have a pointer to a structure containing a function pointer, making in total less than 30% of all structs that are considered prime targets for exploitation. Additionally, the kernel developers have honed their

defenses specifically against exploits that achieve control flow hijacking [7, 12], leaving a slew of data-driven attacks underdefended, such as `struct cred` swap attack [23] which bypasses all known defenses. The attacker's search for sensitive data-driven attacks can be illustrated as a search for a needle in the haystack where the latter is also ever-changing. In fact, many Linux kernel exploit authors rely on prior knowledge, when writing Linux kernel exploits. For example, many Linux kernel exploits use their memory corruption capability to corrupt `addr_limit` field [27] or `modprobe_path` [21]. The `addr_limit` overwrite became a popular method ever since it was first publicized in 2010 [31] due to its simplicity and the level of control it provides. Historically, it was reused in 2014 by a variant of the infamous Towelroot [28] exploit in 2019 by a 0-day [33]. In 2021 a study by Google Project Zero team showed that `addr_limit` primitive is used in 3 out of 7 Android kernel exploits seen in the wild [2]. In 2020 kernel developers completely removed this attack surface [32] rendering all the exploits that relied on it obsolete, but Android is usually not running on the latest Linux kernel, which is why it was still exploited in the wild.

The process of finding these sensitive fields in kernel structures involves expertise in specific modules of the Linux kernel code base. The attackers usually rely on each others' expertise to find such primitives and simply reuse them. As mentioned above, if the defenders introduce new technologies, they may render existing exploitation primitives useless or introduce new ones. The attackers are then left to manually go back to reading the source code and find a new one or fix the existing one, thus requiring significant manual effort. In fact defenses have been proposed to sensitive fields that can be directly used to reach exploitable states [6, 11, 39], however they all lag behind in deployment since they are late responses to what the attackers find and decide to publicize.

In this paper, we introduce SCAVY, a framework to automatically search and find new exploitation primitives in Linux kernel that can reach privilege escalation. Our work starts by defining the exploitable state in the context of a local unprivileged attacker trying to execute with escalated privileges. Automating the process of finding primitives that lead to exploitable states requires solving 2 major challenges.

First, the search space is unique from that of bug explo-

ration frameworks [3, 5, 22, 38], since we are not limited in exploring capabilities of a single given kernel bug. Additionally, unlike Syzkaller [34] SCAVY is not trying to find new bugs in the Linux kernel. Instead we are searching deep in the weird machine states of every hypothetical *single-field* structure memory corruption for exploitable states. At runtime for every program scheduled to execute SCAVY systematically chooses a field a kernel object to corrupt and does a multi-step corruption-based fuzzing, focusing on maximizing the usage of the corrupted field. We modified the execution step such that it executes the program marking the memory area of interest and recording all the code that loads from this area of memory. This first step is necessary to inform the fuzzer of the number of important code blocks that access the memory of interest. The fuzzer then uses this information as code coverage which it normally tries to maximize. Then the same program is re-executed to check if the corruption will cause a crash. If it does, the default fuzzer behavior will dump a crash log that can be investigated for its potential to reach the exploitable state. During this stage our fuzzer produced 3863 PoC programs whose respective corruptions cause various crashes.

Second, SCAVY is detecting complex exploitable states that are not simply write-what-where or Instruction Pointer hijacking. Our system detects exploitable states that allow the attacker to read or write into resources, such as files and memory that normally they should not, such as `/etc/passwd` or `/etc/shadow`. We argue that these are considered exploitable states since they allow the attacker to escalate their privileges, as it's demonstrated by PEASS-ng [29], a popular script that searches misconfigurations in the system that allow read/writes into sensitive resources. It is particularly challenging to detect these states since the fuzzer usually runs as the root user and the number of crashes that need to be investigated is orders of magnitude larger than any prior work, therefore we cannot resort heavy techniques such as symbolic execution or systemwide taint tracking [41]. As mentioned earlier, SCAVY needs to analyze the exploitable state reachability in 3863 PoCs, while KOOBE [3] and FUZE [38] work with less than 50 PoCs. Our solution is to detect privilege de-escalations. The rationale is that a corruption such as an overwrite on the `uid` can cause the program to no longer open `/etc/shadow`, it can also cause the opposite effect. Therefore, in the case where opening these files or performing any root-only operation would fail, SCAVY will detect this failure and show that it was caused due to our memory overwrite. The detector of SCAVY showed that from 600 randomly picked PoCs it detects twice as many of them to reach exploitable states compared to prior work.

After running the fuzzing step for 7 days, SCAVY produced 3863 crashing proof of concept programs corrupting 2811 unique fields across 139 kernel structures. The exploitable state detector returned positive on 955 PoCs. After manually analyzing some of the fields with the largest number of detected PoCs we confirmed 20 of them to lead to exploitable states. Out of them, 3 were considered common knowledge and are still available in Linux version 5.15, although maybe more difficult to corrupt. SCAVY found 17 other fields that were not known to be used in any kernel exploit. For ex-

ample we found that `struct file` has a pointer to a `struct address_space` object. The kernel uses this object to determine the pages corresponding to the file. If the attacker overwrites this pointer to point to a different address space they can read and/or write into that other space as if it's their file, because the access flags are checked on the original struct file.

In summary, we make the following contributions:

- We propose an extensive definition of exploitable states for kernel exploitation, which result in privilege escalation. With that, we designed a system that tackles all the challenges of systematically corrupting fields in kernel objects and detecting the resulting exploitable state.
- We implemented SCAVY, a framework that leverages Syzkaller and static instrumentation to track and systematically corrupt various kernel objects until we can reach an exploitable state.
- We evaluated the effectiveness of our system in finding 20 highly corruptible fields in 12 kernel structures that were not previously used to achieve privilege escalation. Out of those, SCAVY found 3 previously known sensitive fields.

This paper is organized as follows. In Section 2 we motivate the problem and where SCAVY stands in the grand scheme of things. In Section 3 we introduce our definition of exploitable state and show the design of SCAVY that allows it to search and detect those states. In Section 4 we show how we implemented SCAVY and why we used the technologies we did. At last, in Section 5 we evaluate the effectiveness of SCAVY in exploring the search space and the benefit of our definition of exploitable state.

2 Related Work

In this section, we highlight important prior work and provide background information on the whole pipeline of Linux kernel exploitation and where our work stands in the grand scheme of things.

The typical workflow of finding and exploiting a bug in the Linux kernel has 5 general stages: (1) finding a bug and a way to trigger it to a crash, (2) exploring its other crashing capabilities, (3) achieving a consistent corruption, (4) bypass defenses, (5) combine with other capabilities to have a non-crashing exploit with privilege escalation. As described above, SCAVY is used to automatically find corruptible fields of kernel structures, to expedite the step 5 of the exploit writing. Thus, SCAVY may aid the existing automatic bug exploration tools [3, 22, 38] in finding more exploitable states.

Finding a bug There has been numerous papers proposing efficient ways to get the Linux kernel to crash. This is a two-pronged approach, which requires a program that will issue system calls. For the first, the most popular fuzzer is Syzkaller [34], followed by others that make use of more complex techniques to achieve higher code coverage and number of crashes [10, 16, 18, 20, 35, 43].

To narrow the distance between the bug and the crash they all rely on the second prong, the sanitizers. These are tools

that detect illegal operations, such as an out-of-bounds read or write, use-after-free, race condition etc [14].

Our work makes use of the capability of Syzkaller to maximize code coverage and the large number of handwritten templates that it uses to explore known ways to interact with a specific kernel structure’s field. As stated earlier, we modify the signal that Syzkaller uses so that it prioritizes the code coverage for the code that hits the memory area of interest.

Exploring the bug After a bug is found, one needs to evaluate the capability of the bug. Based on the bug, prior work have proposed methods to further investigate its capability and severity. To investigate the other capabilities of an out-of-bounds write Chen et al. [3] introduce a capability guided fuzzing that explores larger capabilities and larger code coverages based on an arbitrary probability on which to explore. When a new capability is detected they employ symbolic analysis to check whether it can overwrite an object’s field which would lead the system to an exploitable state. In a similar fashion, Wu et al. [38] explore the use-after-free bugs. Unlike the earlier work, exploiting the UAF bugs imposes different challenges, such as detecting the line of code that causes the dangling pointer and to correctly exploit the time window of a race condition. In the final step FUZE also detects whether the system can reach an exploitable state, with the added step of accounting for some kernel defenses like SMEP and SMAP. Both the aforementioned papers skip steps 3 and 4 and jump to a stage before step 5. Similarly, we also disregard those 2 steps.

In a more recent paper, Lin et al. [22] focus entirely on exploring multiple crashes of the same bug. Unlike the prior papers, they don’t directly assess whether the new bug reaches a specific exploitable state. They stop the search once the new bug is something that KASAN flags as UAF, OOB access or double free, because it’s now more likely to be exploitable. To find new crashes GREBE analyzes the original PoC to extract the objects involved in the crash. Then it prioritizes the exploration of code paths that allocate and use those interesting objects.

Our work is similar in nature to GREBE, because we also modify our fuzzer to search in an object-driven fashion focusing code that accesses the object of interest. However, unlike GREBE we are not focused on 1 single object throughout the runtime of the fuzzer. Our fuzzer dynamically switches focus based on the object it chooses to corrupt for each program it runs. Additionally, we have a more fine grained targeting where we are interested in code that touches the target field (up to 8 bytes) inside the object of interest to prioritize specific code that loads the corrupted value. Additionally our definition of exploitable state encompasses a larger range than FUZE or KOOBE. Unlike the 3 papers, SCAVY assumes the attacker has a certain memory corruption capability and doesn’t try to explore real bugs in the Linux kernel. Therefore, one can consider our work as a middle step between bug behavior exploration and assessment of it’s ultimate exploitability, meaning the exploitability of the best capability the bug may offer. In step 5 we explain how our system helps supercharge the 2 earlier bug exploration tools.

Consistently corrupting the target Once the attacker has confirmed that they can trigger a bug and exploit it, they now

need to make it work on real-world systems. This means that the exploit should be able to massage the heap layout such that the bug works on the correct victim object. Nikolenko [25] shows a method of heap feng-shui that extends to all slab caches and thus can be recycled for multiple kernel exploits. Chen et al. [5] propose SLAKE, a system that automatically find system calls to allocate objects of interest to the exploit author’s bug. Additionally, they show what system calls the exploiter would need to call to get the desired heap layout. Zeng et al. [42] take a different approach. They empirically test commonly known methods of heap layout manipulation and other exploit stabilization methods and report the success rate of each of them.

We consider this step out of the scope of our paper, because we assume that the exploit author has the capability of always performing the memory corruption to his object of interest. SCAVY is designed with the ability to consistently corrupt the same target object regardless of the heap layout.

Bypassing defenses While the earlier 2 papers try to reach Instruction Pointer hijacking, considering the system exploitable, Wu et al. [37] show that the attacker can bypass all modern defenses with any level of IP hijacking primitive by chaining multiple kernel-side ROP gadgets. Exploit author have also come up with standardized methods to breaking certain defenses, such as KASLR [13, 17] or SMEP [26]. Additionally, to help the exploit author with choosing what kernel objects to use for data exfiltration Chen et al. [4] propose a systematic method to find all objects that have variable length which can be changed through an overwrite and may allow OOB reads. Zou et al. [44] demonstrated it’s practicality in a real exploit. To bypass SMAP, one may direct a corrupted data pointer to point to kernel’s *physmap* region where they control objects using *ret2dir* [19].

This step is considered out of scope for our work, since bypassing kernel defenses is mostly dependent on what information the attacker can exfiltrate from the kernel memory, which is different for each bug. As we show in Section 5.2 for some fields that we found, the attacker is assumed to have a primitive to leak the required value they need to overwrite them with.

Reaching the privilege escalation The pinnacle of a local kernel exploit is to reach privilege escalation. This could be done by having arbitrary code execution in kernel context or by simply escalating the the current user’s privileges to root. For the latter exploit authors have created standardized tools [29] to automatically search for misconfigurations in the running system to achieve privilege escalation. Meanwhile, for the earlier, prior work [3, 5, 38] have a definition for reaching the exploitable state. KOOBE considers the system exploitable whenever their greedy algorithm can find that there are enough capabilities to modify fields in objects of interest. To identify these fields they focus on (1)function pointers, (2)data pointers pointing to function pointers and (3) non-pointer fields. Finding the first 2 is straightforward, while the third type is found manually. In our paper, we propose an automated method specifically to find the type 3 fields. Thus, KOOBE may benefit from SCAVY by being able to reach exploitable states for more OOB kernel bugs.

FUZE relies on symbolic execution to track the corrup-

tion across multiple execution paths. If the execution reaches a state where the attacker controls the instruction pointer or an instruction that moves attacker controlled data and/or to attacker controlled destination, then they evaluate if it's enough to bypass SMEP/SMAP. If it does then it's deemed exploitable. We show in our paper that this is a very limited definition of an exploitable state, since an attacker doesn't need to directly use the corruption to execute arbitrary code in kernel space. They can simply change the address mapping, *uid/gid* or credentials [23] to escalate their capabilities.

Defenses As mentioned earlier, attackers have known ways to break certain defenses, or massage the heap. Additionally, there are known kernel objects and fields to corrupt to get privilege escalation. For example, one can corrupt the freelist pointer of a slab allocator to direct the next allocation into user space. To protect this sensitive field defenses have been proposed, such as XOR'ing, moving it at a larger offset from object start [6], poisoning the area around it [15] etc. However, there are plenty of other fields known [39] that don't have such protections. SCAVY provides a systematic way of finding these fields and new ones.

2.1 Motivating examples

In this section we take an example exploit that breaks away from the traditional function pointer overwrite method and two another similar methods that have been found in real world exploits.

CVE-2021-26708 is a race condition bug that provides the attacker with a write-after-free primitive in *kmalloc-64* cache, which allows the attacker to write 4 bytes with data they control. After chaining multiple primitives the author uses an arbitrary write primitive twice to write zeros to the 4 *uid* fields in *struct cred* [30]. Then the attacker can proceed executing as if they are the root user.

The other method to exploit *addr_limit* field. This field was patched out from the kernel since 5.10.0-rc3 [32]. However, Google Project Zero team have shown in that this attack method, first published in 2010 [31], is still popular for Android exploits in the wild as of 2020 [2, 33].

Another non-function-pointer attack involves attacking the slab allocator, kernel's heap allocator. In Figure 1 we illustrate how this exploit method works. An attacker allocates an object that has an overflow vulnerability, next to a freed slot. The freed slots have a pointer pointing the slab allocator to the next free slot it can use, similar to a singly-linked list. Then the attacker causes an overflow which corrupts the next pointer. When a legitimate kernel routine calls *kmalloc* the next pointer will instruct slab allocator to allocate now from an attacker controlled memory space. In this memory the attacker has read and write privileges to all the allocated objects. This attack is very easy to perform and recyclable across multiple vulnerabilities, therefore defenses have been put in place, such as XOR'ing the pointer with a secret value and moving it in the middle of the object to protect it from small overflows.

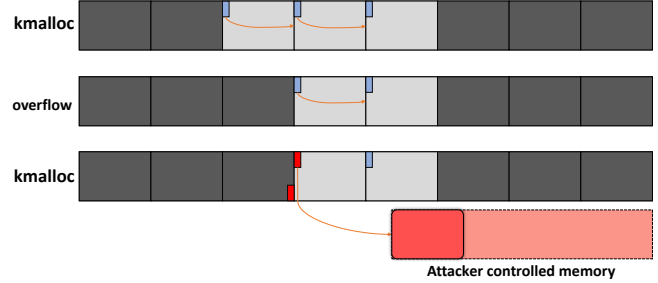


Figure 1: Visualizing slab freelist corruption

As we illustrate there are multiple sensitive fields in the Linux kernel that allow an attacker to escalate their capabilities without having to modify *refcount* or any function pointer. The *uid* field is still not very protected against page-level overflows. For example, an attacker with the capability to write 8 pages as in CVE-2022-27666 [44] in kernel memory can force the buddy allocator to allocate pages for *struct cred* objects in the area they control and overwrite the uids as in the example above.

3 Design

In the following sections we define the search space and the design decisions made to satisfy the requirements for effectively searching and detecting exploitable states. We highlight various tradeoffs and the reasons for deviating from prior work.

3.1 Exploitable states

In this paper, we assume the threat model of a local unprivileged attacker trying to use kernel vulnerabilities to escalate privileges. This threat model is common in malware that need to perform privileged operations, such as establishing persistence [1]. The threat is more common nowadays in cloud computing, Android and Docker. The ultimate goal for the attacker is to elevate their privileges. A well known tool to automate this process is PEASS-ng [29]. It is a collection of scripts that search for potential misconfigurations in the host that can allow an attacker to modify privileged resources. For example, PEASS can find an */etc/passwd* file that allows writes from unprivileged users. This misconfiguration can be exploited to add a new user in the system that has *uid* 0 and then log in as that newly created user. Another example is if an attacker can read */etc/shadow*, they can get the hashed passwords of each user and work to crack those hashes and log in as root.

The two examples above show that the attacker doesn't need to reach IP-hijacking primitive to exploit a system. Therefore, we take inspiration from PEASS-ng to compile an exhaustive list of all exploitable states that are required for the privilege escalation.

- Write to
 1. root-owned files

2. root-owned memory
3. read-only files
4. execute-only *setuid* files
5. read-only directories

- Read to

1. unreadable root-owned files
2. unreadable directories
3. unreadable root-owned memory
4. kernel memory

3.2 Overall design

In this section, we put all the pieces together and demonstrate the end-to-end pipeline of SCAVY. In Figure 2 we illustrate the whole system from the detection of the exploitable state backwards from *Detection* (box in blue) to *Static instrumentation* (box in green).

3.2.1 Exploitable state detector

```

1  int SPRAY_FILES = 100000;
2  int res[SPRAY_FILES];
3  for (int i = 0; i < SPRAY_FILES; i++) {
4      res[i] = open("/etc/passwd", O_RDONLY);
5  }
6  int vic = open("/tmp/writablefile.txt", O_RDWR);
7  void * mapping = get_mapping_ptr_from_spray()
8  trigger_mapping_overwrite(<CORRUPTION>);
9  write(vic, "root2:x:0:0:root2:/:bin/bash\n", 32);
10 close(vic);

```

Listing 1: Sample of exploiting mapping pointer in struct file.

```

1  int vic = open("/etc/passwd", O_RDONLY);
2  trigger_i_uid_overwrite(<CORRUPTION>);
3  close(vic);
4  int vic = open("/etc/passwd", O_APPEND);
5  write(vic, "root2:x:0:0:root2:/:bin/bash\n", 32);
6  close(vic);

```

Listing 2: Sample of exploiting i_uid pointer in struct inode.

To detect the exploitable states, we need a framework that can detect unallowed executions of system calls or wrong return value of allowed ones. For example, a non-root user should not be able to successfully call *seteuid*(0) (ie. the syscall should return *-1*). Additionally, the buffer output of a *read*() syscall, should it succeed reading, would be the same value across 2 executions. Based off our exploitable state definition, system calls that can reach them are *read*() and

write() on resources such as files, sockets, etc. We say that the system is in an exploitable state caused by the memory corruption when we detect a deviation in execution between the normal execution and the one where we corrupt the kernel structure.

One straightforward way to detect deviations it is to run the crash-causing program through symbolic execution and make the corruption a symbolic value. Then throughout execution track if the symbolic value can cause the write to happen outside the writable memory of the program or if the corruption can lead the code flow of the syscall to return *-1*. We decided to not pursue the heavy symbolic execution method, because the corpus of crashes, produced in the *Search* stage in Figure 2, has 3863 crashing proof of concept programs and analyzing each one with a symbolic executor would take too long. Instead, we execute the PoC three times. First, without performing the corruption, then we have the PoC perform a controlled corruption where we copy the value of the field of interest from that of a root process' kernel object and then we overwrite with a random value.

We leverage KCOV¹, a kernel driver that outputs the code coverage of a syscall, to roughly estimate if the execution is going on the correct path. Basically, we run the PoC without corrupting its field of interest, then we collect the code coverage for all the *read* and *write* system calls (including all their vectorized alternatives). Then we re-execute the PoC where we corrupt the field of interest with the same value as a root object's field. Then we compare the return values, buffer values, code coverages of the 2 executions. If the code coverage is has a Jaccard similarity greater 0.9, the return values are the same and the values in the buffers differ, then we say that the privileged read/write has happened. In some cases the corruption may not directly affect the read/write operations, such as when overwriting the uid or the setuid bit.

Of course, we need root-owned resources to target. We noticed that the default Debian stretch image of Syzkaller has an Apache service running. We use our object tracker to target it's resources. Additionally, we create a synthetic root process that opens privileged files, directories and sockets. This approach is based on the SELinux regression test suite². Since there might still not be enough unique kernel objects, we track all processes until PID 250 (roughly all system processes started on boot).

Across the 3 executions we collect, the return values, the buffer values, the code coverage and the corruption hits only for syscalls after the corruption. We feed these 4 values for all 3 executions to the privilege escalation detector, which will flag a PoC if significant deviation is detected. Since by default the PoC is supposed to run in root privileges, the detector looks for privilege de-escalation, because if a corruption can cause de-escalation it can also cause escalation. The deviation from return value indicates if the syscall failed and the deviation from buffer values indicates if the read data is coming from a different source.

We noticed that in many cases a corruption doesn't directly

¹<https://www.kernel.org/doc/html/latest/dev-tools/kcov.html>

²<https://github.com/SELinuxProject/selinux-testsuite>

lead to an unsafe read or write and the produced PoC doesn't have any of those syscalls. Therefore as shown in Fig. 2, we have an enrichment step for all the programs we analyze to make sure the syscalls of interest appear. We add `read`, `write` and `fsync` system calls on all the resources created before the corruption injection. Additionally, we insert illegal syscalls such as `seteuid(0)` and `open("/etc/shadow", O_RDONLY)` which in case of privilege de-escalation should fail. We noticed that for 1422 crashes there is only 1 resources and 1025 have 2 resources while the rest of the crashes have more than 2 resources. For example in Listing 2 our detector would return positive, because line 4 would fail if we overwrite uid with non-zero value.

In Listings 1 and 2 we demonstrate two proof of concept samples and how SCAVY would detect them. It is well known that if an attacker can write into `/etc/passwd` they can create a new root user and log in as that user. In Listing 1 the attacker is spraying kernel memory with `struct` files that have an address space pointer called `f_mapping` that points to the pages in memory where the content of the file are allocated. If the attacker can leak this pointer they can overwrite it into their own struct file (created in line 6). Then the attacker will have the same permissions to operate in that address space as if they were writing into the `/tmp/writeablefile.txt`. In a successful exploitation the write at line 9 would return 32, the size of what was written. However, if the attacker overwrote a random value or a 0 at line 8, the `write` would fail and the kernel would panic. In this case, the write would return -1. In a normal execution with no corruption the write would still return 32, therefore to detect the fact that the memory corruption caused the exploitable we need to execute this code 3 times. Once with disabling line 8, once with line 8 having 0 as a parameter instead of `mapping` and once as it is shown in the listing. Then compare if return value of line 9 in execution 2 is any different from that in execution 1 and 3, while the latter 2 are the same. In Listing 2, the exploitable state is then we are allowed to write in an `/etc/passwd` file descriptor opened in append mode. Normally the proof of concept code produced by the fuzzer depicted in the Search step in Fig. 2 would run in root and so the code that we want to detect would just run as intended if there is no corruption. The `open` in line 4 would return -1 if in line 2 we overwrite `i_uid` with any value other than 0. Therefore if we execute the same code 3 times as we did above we would detect a deviation because the return value in execution 3 is different from that in execution 1 and 2.

3.2.2 Searching for corruptions

In order to systematically explore memory corruptions in all the fields of all the kernel structures we need a system that is able to allocate all the kernel structures. The simple way is to run a benchmark tool, such as Phoronix³. However, the major drawback with this method is that the corpus of programs may still not cover all possible interactions with the allocated objects. A fuzzer is a better approach because when maximizing code coverage it will hit many unique allocation sites and

unique code paths that interact with those objects. There are a major challenges with this approach. The fuzzer will take too long to reach code that hits specific corruptions in each kernel object since it only cares about total code coverage. For example, the fuzzer will open a file, then open a socket instead of writing to the file it just opened, because opening a socket hits more code.

Our solution is to modify the mutation phase of the fuzzer. The intuitive approach, is to follow the example of GREBE [22] and separate allocator system calls from utilizer ones and have the mutation phase choose utilizers. This would work well if we had a PoC to start from that already has the code to allocate the vulnerable object, as in GREBE's case. In our case, we start from nothing so we need to utilize the built-in manually written templates of Syzkaller to allocate all reachable kernel objects.

We designed our fuzzer with the ability to diversify its corruptions and to prioritize code paths that hit those corruptions. Unlike the default fuzzer that relies on code coverage, or GREBE's version that relies on critical object's code coverage, ours relies also in 3 parameters: code coverage, critical object's code coverage and corruption code coverage. The critical object's code coverage is a special code coverage that only considers the code that operates on the object of interest. Unlike GREBE our critical object's code coverage is dynamically computed for each running program since they have different object to corrupt. Meanwhile, corruption code coverage is the code that directly reads the corrupted field inside the object of interest. The more the corruption is read the deeper in weird machine states we are computing, so we incentivize the fuzzer to maximize this type of coverage. To prioritize, this type of coverage we generate 20 synthetic IPs per corruption hit to make the code coverage guided fuzzer prioritize seeds that have high number of corruption hits.

Specifically the fuzzer executes each program in 3 stages, unlike the single shot execution of a regular fuzzer. First, we run the program sequentially from its 3rd to its 10th system call. If a program is smaller than 3 system calls we discard it from executing any further. If it has less than 10, we run until the last syscall. During this phase the fuzzer records the objects allocated for each syscall's execution. From all the objects it picks the one with the field that has the least number of corruption hits and marks the first syscall that allocates that object type.

In the second stage the fuzzer modifies its code coverage filter such that every syscall after the mark only contains code coverage from source code files that use the object of interest. This usage info is fed to the fuzzer in a dictionary form from an earlier static analysis step, which we don't include in Fig. 2. Note that this filtering doesn't affect the syscalls earlier than the mark, because we still want the fuzzer to explore new code spaces to allocate new object types. The fuzzer then re-executes the entire program, but at the mark it will immediately set up a hardware breakpoint to track the memory accesses of the field of interest. The fuzzer knows the offset and size of the field of interest, because it's fed this information in dictionary format parsed from a run of `pahole` on the instrumented kernel's binary. By the end of the execution of this stage the fuzzer knows the filtered code

³<https://www.phoronix-test-suite.com/>

coverage and the number of times the interesting field was read. If the number of hits or filtered coverage is larger it will add the program to the corpus for further mutation steps.

In the third stage the program is re-executed, but now the fuzzer will also overwrite the field with random data and collect the same data. If the program crashes the kernel the fuzzer will store the crash for further analysis. It's important to note that normally the kernel wouldn't crash if the *uid* in *cred* is overwritten to a random value. In cases when a crash doesn't happen, even with an enabled KASAN, the fuzzer compares the filtered code coverage from second and third run. If the size of unique IP values is different, then the fuzzer will produce a warning message, which adds program into the crash corpus. Since, by default the fuzzer runs as a root user, the overwrites will downgrade the privileges, which would reflect with further syscalls failing. We explain the implementation details in Section 4.2.

3.2.3 Static instrumentation

Tracking object allocations It is challenging to detect the kernel object allocations for each system call. GREBE have started their exploration from a PoC and so they could afford to run expensive taint analysis to find out the allocated object type. Meanwhile in FUZE the authors don't need to know about the object type and in KOOBE they only need to know the object type after the analysis is complete and a capability match is found. Therefore, they can all afford slow computation since there is only a few objects of interest. In our example, we need the fuzzer to know at runtime exactly what objects are allocated and what's their data type and when are they freed.

One way to do this is to hook *kmalloc* and dump the stack using the built-in *dump_stack()* function and then determine the allocated object based on the line of code in the stack trace. Basically, after static analysis one can flag all lines of code that will reference the newly in a non-void pointer and if that line appears in the stack trace then we know the object type. However, *kmalloc* is called from every running process and dumping the stack oversaturates the *dmesg* pipe. Another way to do this would be to BPF filter and selectively dump stack from *kmalloc()* calls of the fuzzer process. Unfortunately, the kernel limits the BPF logging buffer and the bottleneck saturates too fast when a fuzzer is running. Not to mention that it would still rely on static analysis to determine which line of code allocates which object type and static analysis has its limitations in the kernel.

Our solution makes use of LLVM's *CastInst*⁴ pseudo-instruction. This instruction allows LLVM to always have an up-to-date data type for its variables. The pointer casting, as also highlighted in Linux kernel coding guidelines, is unnecessary since C will automatically convert pointer type when assigning to a variable [24]. Therefore, LLVM will always have a *CastInst* when the void pointer is assigned to another non-void pointer variable.

Our LLVM pass adds a function call to every *CastInst*. This call is to a function we create, with 3 parameters, (1)

the pointer address, (2) data type before typecast, (3) data type after typecast. To make sure we only instrument *CastInst* of heap memory and not stack variables we only instrument *CastInst* from a pointer type to another pointer type and the destination type is a struct. The second rule is important since corrupting a character in the memory of a *char ** variable wouldn't lead to any meaningful exploitable state unless it's the last character and the attacker would read until the first occurrence of a NULL value.

We run 86% of the kernel code through this custom compilation step that instruments the typecast instructions and observed that instrumenting some startup and interrupt code causes the kernel to panic on boot. Since the instrumentation step happens before any optimization we believe that it corrupts optimization-sensitive code. Linux kernel cannot compile with optimization level 1 or 0. To make sure we are not missing data types from any kernel object we ran the fuzzer in default code-coverage mode and recorded all allocations and how many of those could be resolved to a struct type. On average around 65% of the allocations were resolved to a kernel structure across all system calls with *sys_select* only resolving 31% of its allocations. When investigating further we found that many of those allocations are *char* arrays, which would not be useful when trying to reach privilege escalation. After running the fuzzer for 24 hours we detected 139 unique structure types.

During the fuzzing step, we leverage, Kprobe, a tool that allows root users to insert breakpoints in the kernel functions and print their arguments. In our case Kprobe will fire at our inserted function and print out its 3 arguments, the pointer value, the data type before the cast and after the cast. Additionally, we use Kprobe to print the return value of *kmalloc()* and *kmem_cache_alloc()* as well as the argument of *kfree()*. After adding the 4 probe points we can ask Kprobe to fire on specific process PIDs so that we only track fuzzer's operations. With the 4 Kprobe points the fuzzer can see at runtime what memory locations are alive (allocated, not freed) and relate the data type that memory has been cast to.

Corrupting memory and tracking it To manually corrupt kernel memory is not a challenging task. Basically, we only need to insert a kernel module that can be controlled through the *ioctl* interface to corrupt a memory location from what the fuzzer parsed earlier. The challenge is in how to track where that memory is used. A great way to do this is to run a taint analysis that maintains the taint of the accessed corrupt memory location across multiple function calls. One can instrument *qemu* to do taint tracing [9, 40], but this would slow down the fuzzing significantly since we would need to track all memory loads, stores and any instruction that uses their values. Since the fuzzer would require many executions until it reaches memory accessor code that uses the corrupted memory it would mean that most of the time the taint tracker will uselessly run.

Instead, we leverage hardware breakpoints. In x86 the kernel uses up to 4 registers to set up 8 byte hardware breakpoints. A hardware breakpoint is a CPU feature that allows it to raise an interrupt whenever it's about to load and/or store into a memory location. Since it's a hardware feature it poses no performance penalty. Whenever the hardware memory break-

⁴https://llvm.org/doxygen/classllvm_1_1CastInst.html

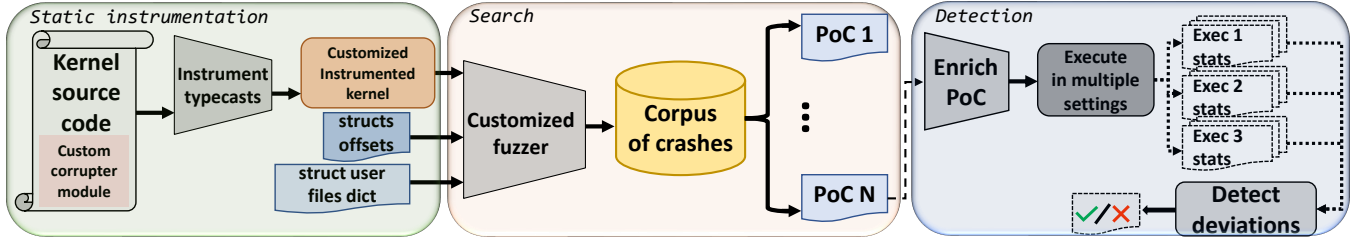


Figure 2: Overall system design

point is hit it will print out the *RIP* value in *dmesg*, which the fuzzer parses. The only drawback of this method is that if the corrupted value is copied into a local value and the now-corrupted local variable is used somewhere in the function we won't be able to track it. This is not a problem since we don't use the corruption hit alone when guiding the fuzzer.

4 Implementation

We wrote SCAVY in Go (≈ 1000 LoC), C (≈ 200 LoC), C++ (≈ 1100 LoC), and Python (≈ 600 LoC). Go is used to customize the Syzkaller. In C we wrote the in-tree kernel driver used by the fuzzer to corrupt and set up hardware tracing. C++ is used to write the 3 compiler passes, two of which produce the input to the fuzzer and the third to instrument the kernel typecasts, as shown in the green box in Figure 2. Python is used for the analysis step to parse the program from crash logs, enrich it and run it 3 times, parse its 4 different output types and detect the deviation.

4.1 Compiler instrumentation

The input to SCAVY is the modified kernel source. For this paper we chose to use the LTS Linux kernel version 5.15.80. First, we add an in-tree driver in the `drivers` folder. We also modify the `Kconfig` and `Makefile` in that folder. Then we simply enable `CONFIG_CORRUPTER_MODULE` in the `.config` file before compilation. When making the kernel we change the `CC` to point to a shell script that calls a python script that makes the decision whether the file should be instrumented or not based on its relative path. The instrumented file is ran through 2 of our compilation passes by `clang`'s `Xclang` plugin with the following command `fcommon -Xclang -load -Xclang <pass.so> -flegacy-pass-manager <input.c>`. The uninstrumented file is compiled with the default `clang`.

The first pass produces a text file that contains the structure types that each function in the file works with. The second pass instruments every `CastInst` instruction whose source is an `int8*` and destination is a struct pointer. The instrumentation step involves adding a function call to an external function defined in our in-tree custom driver. The function takes as arguments the name of the caller function (the one we are instrumenting) the pointer value that is being typecasted, and its new type. To protect the function for any optimization

step we make it change a global variable to the XOR value of the passed pointer and its caller function name's pointer.

4.2 Fuzzing for privilege escalation

Our fuzzer is a modification of Syzkaller. We modified its 3 core components: `syz-manager`, `syz-fuzzer` and `syz-executor`. The manager is modified to load the 2 dictionaries illustrated in Fig. 2. Once loaded they are sent to `syz-fuzzer`. The manager also has a parameter for 2 modes of execution. In the total tracking mode it enables all syscalls and logs out all the objects created. The second mode is the normal privesc exploration one.

Modifying syz-fuzzer The `syz-fuzzer` is the core of our modifications. It is the program that generates and mutates the seeds that will be run by `syz-executor`. We modify its execution step to execute the same program 3 times instead of once as mentioned in Section 3.2.1. For each execution we collect data from `KCOV`, `Kprobe` and `dmesg`. Normally `syz-executor` collects the code coverage from `KCOV`, our modification allows `syz-fuzzer` to collect the allocations frees and typecasts from `Kprobe` and memory hits of corrupted memory from `dmesg`. To collect data from `Kprobe` our `syz-fuzzer` first instructs `Kprobe` to set up its probe points for `kmalloc`, `kmem_cache_alloc`, `kfree` and `instrument_typecast_instruction`, the custom function that does nothing. The fuzzer instructs `Kprobe` to print all parameters of the custom function and `kfree` and the return value of the two allocation functions. After setting up the `Kprobe` point our fuzzer will enable `Kprobe` to run right after spawning the executor. Then it will run a thread in parallel to parse the output logs of `Kprobe` and filter out any logs that are not generated by `syz-executor`. To parse `dmesg` the `syz-fuzzer` will spawn another thread in parallel.

Another modification to the `syz-fuzzer` is in the way it communicates with executor. If the seed has a corruption after a syscall (ie. at syscall 6) the executor is instructed to pause execution and tell the fuzzer to perform the corruption. This step is useful for execution 2 and 3 where the fuzzer collects memory hits from `dmesg` and filtered code coverage. For the filtering we utilize the built-in filtering functionality of Syzkaller. The two differences are that we change the filter on the fly for each structure type that is being corrupted and only apply it to coverage collected after the corruption.

Last addition is the comparison of execution 2 and 3. If the 3rd execution, in which the fuzzer overwrites the target field in the kernel object with random data, does not crash its

filtered post-corruption code coverage is compared to that of execution 2. If a deviation is detected the fuzzer will `printf` a line starting with `WARNING`, which triggers the default log parsing in `syz-manager` which assumes the kernel crashed and adds the log to the crash corpus.

Modifying `syz-executor` We first disable the fork server which makes it certain that after executing the requested sequence of syscalls the executor terminates freeing all the allocated kernel objects. Then we modify the way it executes seeds that have a corruption. As mentioned earlier, the executor replies back to the fuzzer that it has reached the target syscall and waits for another message from the fuzzer to continue with the rest of the sequence until it's done and the coverage it collected from `KCOV` is sent back as a reply to the fuzzer.

4.3 Detecting privilege escalation

After having a directory full of crash folders we need to vet out the ones that are not reaching privilege escalation. In this step we borrowed part of code from `FUZE` which streamlines the interactions with `QEMU`. We modified the initial codebase to run in Python 3.6+. Before enriching the PoC there is parser component (216 LoC Python) we did not draw in Fig. 2. This parser goes through the directory of crash folders and parses the crash description and every program that was executed and logged. In the case of our synthetic crash we had 83 different logs in the same crash hash, the script parses each log file and makes it easy to access each executed program for the next steps. For each log file there are multiple programs executed before or even after the one that caused the crash since some crashes are not fatal. To accurately find the one that caused the parser uses a combination of regular expressions.

Enriching the PoC After parsing the `Syzkaller` representation of the syscall sequence, we get the number of resources created before the corruption. As we observed in Section 3.2.1 there most of the programs have only 1 resource before the corruption. We insert `read`, `write` and `fsync` for each of the resources. For all the syscalls we print out their return values and the buffer value in case of a `read` syscall. These are read by the deviation detection module, the last piece in Fig. 2.

Adding corruption triggering code Once we have the sequence of system calls we use the `syz-prog2c` utility to generate the C code. However, we also need to add into the generated code the part where the memory corruption is injected. We wrote a library and included it as a `.h` file (883 LoC C++) in the generated C code. The library spawns 2 threads along with the original running PoC code, one that sets up the same `Kprobe` reading and the custom driver interacting code we wrote into the `syz-fuzzer` and another thread that runs as root user and synthetically opens the privileged resources. The first thread tracks the objects created by the PoC, the synthetic root process and the native root processes with `PID` ≤ 250 , as we also mentioned in Section 3.2.3. The library code uses pipes to communicate with the original C code to make sure the corruption is inserted at the same spot it was observed in `Syzkaller`'s logs. It is set up to introduce the least amount of extra code in the generated PoC's code and be

easily modified when one wants to execute PoC in different modes. There are 3 execution modes: (1) execute the PoC without corrupting the memory but only track the accesses to the corruptible field, (2) corrupt the field with random data, (3) corrupt the field with data from the same object type allocated by a root-owned process. The 3 modes are ordered to execute and controlled by the deviation detection module, which collects the same execution data for all 3 modes.

Detecting privileged operations and de-escalation By default `Syzkaller` runs as root, so the PoC is expected to run with the same privileges. If it runs in user mode most of the syscalls will return -1 and nothing will happen. This is why we have 3 executions for each PoC. Since a random value overwrite will likely change the root values, ie. the uid from 0 to non-zero, we are able to detect de-escalation. This module is written in Python (600 LoC) and uses part of `FUZE` code to spawn and control the `QEMU` VMs and parse the syscall return values, code coverages and buffer data from the execution of the PoC.

5 Evaluation

We evaluate `SCAVY` in two fold manner. First we evaluate the effectiveness of the fuzzer to exploring the crashes that may lead to exploitable state. Then we evaluate the ability of the exploitable state detector to detect known methods of exploitable state as well as demonstrate new methods. Additionally, we evaluate our claim of having a broader definition of exploitable states, by comparing our detector with `FUZE`'s symbolic execution detection on 600 randomly selected PoCs. All the experiments are conducted in on a desktop running Ubuntu 20.04 LTS with Intel Xeon(R) CPU E3-1245 v6 @ 3.70GHz x 8 and 32 GB of DDR4 @ 2400 MHz of RAM memory.

5.1 Fuzzer effectiveness

We first test the effectiveness of our fuzzer compared to a stock version. Specifically, we test the effectiveness of our design's unique code coverage in exploring the search space as opposed to the stock version of `Syzkaller`. The experiment is designed to test the capability of the fuzzer in reaching deep weird machine states for all kernel structures. We argue that the fuzzer will explore deeper states when modifying the coverage for every program the fuzzer executes such that it only considers code coverage from files that operate on the target struct type and adding synthetic code coverage based on code that loads corrupted field.

In this experiment, we only test the coverage modification component of `syz-fuzzer`. Therefore, we enhance the stock `Syzkaller` with the hardware tracking of memory reads as well as the capability of tracking object allocations and corruption by using the customized driver. We run the 2 fuzzers for 3 days with 1 `QEMU` VM for each. The VMs are configured with 1 core and 1GB of RAM.

Figure 3 shows the total number of times any corruption has been read during the 3 days of fuzzing. This number indicates the number of load instructions executed during fuzzing,

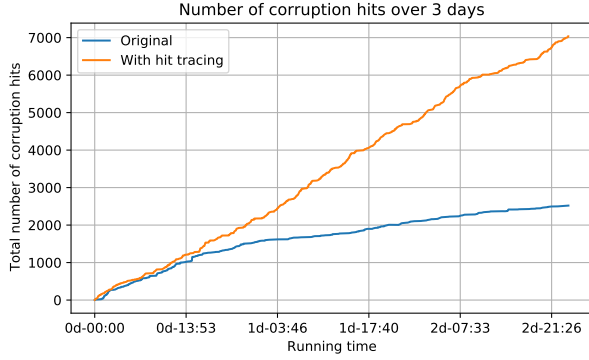


Figure 3: Comparison of total lines of code hitting corruptions

which load from the corrupted memory. The graph shows that for the first 14 hours the 2 fuzzers perform similarly, but the stock fuzzer seems to hit the corruption at a slower rate. This result is expected, because the fuzzer is busy covering code paths that have nothing to do with the corruption. For example, the fuzzer will open a file, then open a socket instead of writing to the file it just opened, because opening a socket hits more code. Our fuzzer will remove the instruction of opening a socket for the program that opens a file because we filter out all the code coverage that doesn't correspond to the object that's being corrupted in that program and during the *minimization* step the syscalls that don't provide any new coverage are removed.

This is better illustrated by the graph in Figure 4, where we show the number of crashes found by each fuzzer. While both fuzzers inject memory corruptions into the least represented structures, the stock Syzkaller doesn't prioritize hitting the memory it corrupts. This can be inferred by the lower number of crashes, suggesting the deprioritization of corruption accessor code over the prioritization of larger overall coverage. On the contrary SCAVY generates more crashes over time either due to unsafe memory access or the sanitizers detecting an unsafe state because the injected corruption was hit and the random value in the corruption was not what was expected.

At last, we argue that even though the fuzzer of SCAVY is focused on corruption hitting code it is not short-sighted in its exploration ability. We measure this by plotting the number of unique struct fields corrupted over time for each fuzzer. In Figure 5 we notice that both fuzzers have the same rate of exploration when finding new fields to corrupt. We expect the vanilla Syzkaller to have a high number of unique fields to corrupt, because when being guided by code coverage the fuzzer will hit more unique allocations sites that allocate unique struct types. However, this graph suggests that our fuzzer doesn't get stuck corrupting the same structs. As we argue in the design Section 3.2.2, our fuzzer is still partially guided by unfiltered code coverage. Therefore, the fuzzer of SCAVY is optimized for both exploration and exploitation, having the benefit of both worlds.

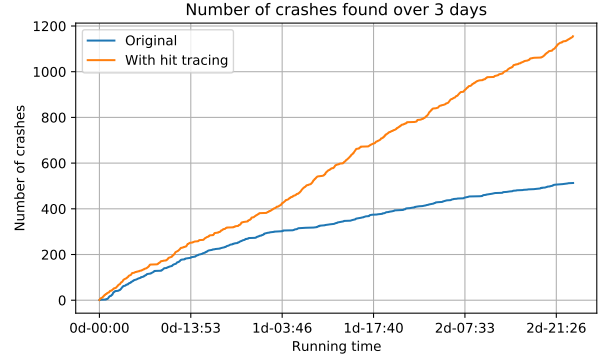


Figure 4: Comparison of the number of crashes found

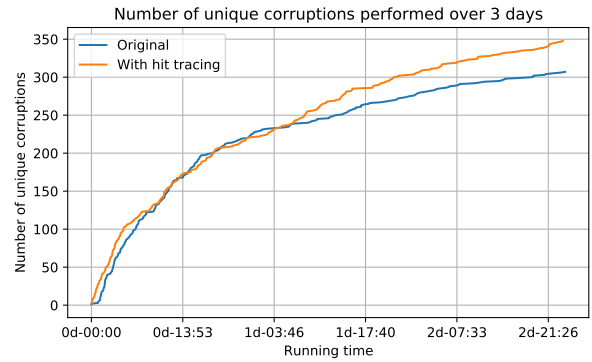


Figure 5: Comparison of the number of unique fields corrupted over time

5.2 Exploitable state detection

In this section, we evaluate the entire pipeline's effectiveness in finding corruptions that reach exploitable state. Our evaluation is two-fold, where we first compare our definition of exploitable state to the common write-what-where and IP hijacking. Second, we demonstrate the effectiveness of SCAVY by showing that it can find prior known fields that lead to exploitable states as well as finding new ones. For this experiment we ran fuzzer for 7 days. In the end our fuzzer corrupted 2811 unique fields from 139 unique kernel structures. The run totaled to production of 3863 PoC programs whose respective object corruptions crash the kernel.

5.2.1 Evaluation of the definition of the exploitable state

For the first comparison we randomly picked 600 PoCs and we ran FUZE's symbolic execution. To make it so that the symbolic executor doesn't get stuck we disabled around 700,000 function call sites that correspond to KCOV, Ftrace, our custom driver and part of KASAN. Specifically, we disabled KASAN functions that poison memory or trace the kmallocs and kfree's in shadow memory, however KASAN is also used to instrument memory loads and stores. We instrument those function calls in *angr* such that the function model remains

Structure	Field	Num PoCs
proc_inode	pid	1
anon_vma	root	8
anon_vma	parent	6
socket_alloc	vfs_inode.i_flags	1
socket_alloc	vfs_inode.xa_head	6
socket_alloc	vfs_inode.i_security	1
socket_alloc	vfs_inode.i_lru	9
socket_alloc	vfs_inode.i_fsnotify_marks	6
pde_opener	c	1
seq_net_private	net	1
kioctx	users.data	1
jbd2_inode	i_transaction	1
shmem_inode_info	vfs_inode.i_security	4

Table 1: PoC with at least 1 primitive found by symbolic execution.

the same, but it doesn’t check the shadow memory. This allowed FUZE to reach the end of the syscall fast instead of getting stuck in an infinite loop. Additionally, to make this experiment feasible we limit the runtime to 1 minute. Then we ran the same PoCs through our privilege escalation detector step, disabling the enrichment step to make the comparison fair.

FUZE’s symbolic executor found 46 PoCs that lead to at least 1 exploitable state. This means that during the unconstrained symbolic execution there is 1 value that satisfies the branches taken to reach an exploitable state. In most of the PoCs FUZE found more than 1 branch leading to an exploitable state, but all of the capabilities found give the attacker controlled write (only the where part being symbolic). We show all the PoCs and the total number of exploitable states found by symbolic execution in Table 1.

Meanwhile, the detection step of SCAVY flagged 276 PoCs for having a deviation that can lead to an exploitable state. We then manually went through the PoCs to verify the validity and estimate the FP rate. Our manual analysis shows that SCAVY by default produces $\frac{131}{600} \approx 22\%$ false positives, however most of the cases are because the kernel tries to deallocate all objects on termination and the corruptions cause double frees or arbitrary frees. For example, when overwriting `inode->i_security` the kernel crashes in `security_inode_free` function, which executes if the pointer is not 0. While these corruptions can still be useful, they are out of the scope of our exploitable state definition, since we are more interested in states that can facilitate privilege escalation. When disregarding these cases we found that deviations happen when we change spinlock values, which indicate to the kernel that an operation is being performed on the resource. These fields don’t pose any risk of exploitability. They make up all the remaining false positive cases, bringing it to only $\sim 9\%$ of 600. In total SCAVY found 91 PoC that were verified to lead the system to an exploitable state, an improvement of 198% compared to baseline. We note the fact that our definition of exploitable state is not a superset of the prior work, since we are omitting IP hijacking and write-what-where in kernel context, for the sake of exploring understudied exploitable

states.

5.2.2 Evaluation of the detection step

To evaluate the detector of SCAVY we ran the exploitable state detection module on all the 3863 PoCs the fuzzer generated. In total the detector returned positive detection on 955 PoCs. We observed multiple PoCs corrupting the same field, therefore we decided to prioritize our manual analysis on the fields with the most positive detections and the ones we found within our expertise to analyze. In Table 2 we show the top 20 fields that were manually verified to lead to privileged operations and the exploitable state that was detected based of our definition in Section 3.1. It’s important to note that the corruptions found represent a subset of what was PoCs we manually analyzed. A full analysis of all the positive detection would require minimal domain knowledge and an analysis with a debugger, but not too much effort since the PoC is already generated.

In the third column of Table 2 we show the states that were detected when we analyzed random PoCs from each corrupted field. In the fourth column we decided to manually modify the PoC to see if another unsafe state could be found. For example, in many corruptions that allowed reads to root-owned files, we tried to perform a write in `/bin/su` and check if it was also allowed. This operation worked in 3 corruptions where the original PoC didn’t reach state 5.

We notice a trend in the results, based off the way we detect exploitable states. The fields with the largest number of PoCs tend to be data pointers such as `f_mapping`, `i_mapping` etc. This may be due to the fact that corrupting these fields will result in a crash no matter if there is a system call following them, because upon program termination an invalid pointer will crash the cleanup stage. And since after we obtain the crash-inducing PoC we enrich it with the syscalls that can reach exploitable states, there is a larger number of PoCs that when enriched will reach exploitable states.

Case study (f_mapping) One interesting file read/write primitive is the overwrite of `f_mapping`. This primitive can be performed in two different ways. First, the attacker can make a root-owned process open unreadable files. For example, spawning many `passwd` processes the attacker can spray the slab memory with `struct file`’s of `/etc/shadow`. Then assuming they have a kernel memory read primitive they can open a readable file (ie. a `/tmp` file) and overwrite its `f_mapping` pointer with the same value he reads from the `struct file` of `/etc/shadow`. Then when issuing a `read()` the data will come from the memory mapping of `/etc/shadow`. Additionally, they can also write into that mapped memory if the original file (ie. `tmp`) is open with write privileges, since the read and write syscalls check other fields in `struct file` for the open flags.

Another simpler idea is to open any of the `suid` binaries, such as `sudo` in read-only and use the corruption to overwrite those files, ie with a call to `system("/bin/sh")`. Then when calling the corrupted `sudo` the attacker will get a root shell and be able to execute arbitrary code.

Case study (vm_file) This example is particularly special. It requires that the root-owned process `mmap`’s a file with

Structure	Field	Detected States	Manually found states	Num PoCs	Known
file	f_mapping*	2, 5	1, 2, 5	65	
task_struct	euid	1	1	5	✓
task_struct	mm*	4	4, 5	1	
task_struct	active_mm*	4	4	1	
task_struct	vma_cache*	4	4	4	
task_struct	files*	1	1	3	
address_space	i_pages*	5	5, 1	3	
vm_area_struct	vm_file*	1	1	1	
inode	i_uid	1	1	1	
inode	i_mapping*	1, 2	1, 2, 5	29	
inode	i_pipe*	4	4	1	
pipe_inode_info	bufs*	4	4	1	
kiocx	aio_ring_file*	4	4	1	
kiocx	internal_pages*	4	4	1	
kiocb	ki_filp*	4	4	1	
key	user*	4	4	1	
key_user	uid	4	4, 5	1	
shmem_inode_info	i_mapping*	4	4	27	
cred	cap_bset	4	4	18	✓
cred	euid	4, 5	4, 5	5	✓

Table 2: **Corruptions found to lead to exploitable states.** The exploitable states are noted as write to root-owned files(1), read to root-owned files(2), write to root-owned memory(3), read to root-owned memory(4), write to execute-only setuid files(5). We mark with a * all field that require leakage of their respective root-owned structures’ field.

MAP_SHARED, which is the default. Then the attacker need to overwrite the *vm_file* with that read from the root-owned mmap’s struct. Then the attacker has to call *msync* to sync the mapping to the newly substituted file. This was found when the fuzzer overwrote the field with 0 and called *memcpy*, which caused it to fail. The detector replaced its value with that from a root-owned file and made the PoC work again. After manually running the PoC in user mode it we performed the corruption to overwrite with 0 and observed that *open* of */etc/shadow* in read only worked, convincing us of reaching the privileged state.

Case study (anon_vma_chain) As it’s described in the source code comments, this structure is used as a hub of all the anonymous virtual memory structures of a mapped, shared virtual memory. These memory areas are treated as copy-on-write, meaning that if a process is to modify them, they will be copied and remapped such that the process doesn’t modify the original memory. Our fuzzer crashed when it corrupted the *rb* field and called *semctl()*, a system call of the System V IPC. While this particular PoC itself doesn’t reach a privileged memory read capability, we noticed the deviation in the code coverage of the *semctl\$GETVAL* call shown in lines 8–10 in listing 3.

6 Discussion and future work

Even though SCAVY is mostly focused on facilitating the exploitation stage we believe a defender can benefit from it too. SCAVY exposes sensitive fields that lead to privilege escalation. With this information a defender doesn’t need to rely on the post-attack analysis to figure out how to defend the sensitive fields. For example, when pointer XOR’ing [6]

```

23:08:35 executing program 0 (corruption
  %struct.anon_vma_chain*+40 (8) at call 4):

r0 = semget(0x3, 0x3, 0x481)
getgroups(0x2, &(0x7f0000000180)=[<r1=>0xee01,
  <r2=>0xee01])
ioctl$NS_GET_OWNER_UID(0xffffffffffffffff, 0xb704,
  &(0x7f0000000200)=[<r3=>0x0])
semctl$IPC_SET(r0, 0x0, 0x1, &(0x7f0000000240)={{0x2,
  0xee01, r2, r3, 0xee01, 0xa0, 0x5f3}, 0xffffffff,
  0x0, 0x0, 0x0, 0x0, 0x0, 0x7})
r4 = semget(0x3, 0x4, 0x40)
semctl$GETVAL(r4, 0x2, 0xc, &(0x7f0000000000)="/" /248)
semget(0x0, 0x2, 0x4)
getgroups(0x1, &(0x7f0000000100)=[<r5=>r1])
r6 = getegid()
getgroups(0x5, &(0x7f0000000140)=[r5, r2, r2, r6, r5])

```

Listing 3: Syzkaller representation of the PoC

was introduced to protect the slab freelist pointer due to the sensitivity of this field to any memory corruption attacks. Similarly a defender may add such protections for the fields that SCAVY exposes. Kernel protections such as KASAN slab redzoning are usually disabled in production due to the intolerable overhead. We believe that instrumenting access to sensitive fields such that redzones are checked only on those can lead to lower overhead, making it possible to deploy in production. The object tracking capability of SCAVY may be used in existing kernel benchmarks to give the analyst a better idea on what objects are commonly being allocated and what

fields are accessed more often. This can lead a defender to deploying a more efficient defense against memory corruptions. Another possible defense is the memory isolation proposed by DirtyCred [23]. This defense though doesn't work against some memory overwrites such as the *uid* overwrite with zeros. In this case the defender may add a new field to the structure's last 8 bytes and fill it with random data, then use this data to XOR the sensitive fields in that structure. This can lead to a memory overhead depending on how often the structure gets allocated. The defender may also move the sensitive fields in the middle or the end of the object, surrounding them with pointer fields to make the overflows less stable, because now the attacker will be overwriting other pointers with illegitimate data causing the kernel to panic on dereference. A promising idea is that of protectable memory, proposed in 2018 [8]. The premise is to add a new function, `pmalloc`, to the allocator which makes the sensitive data harder to find and make them read only once they are populated. The authors propose the "rare write" mechanism to make the data writable only during a brief window frame. One problem they highlight is the lack of users. We hope that SCAVY can highlight some potential users and bring this idea to production.

Exploitation aside, SCAVY proposes a novel object-tracking mechanism and incorporates it with Syzkaller, introducing another dimension in the coverage guided fuzzing. Our methodology can be used for other tasks such as targeted device driver fuzzing or any other fuzzing of the kernel that aims to target specific modules. Additionally, since Syzkaller has a module that attempts to create the PoC that caused the crash it would be important to know all objects allocated during the runtime since in many cases the heap layout might differ. SCAVY can track all object allocations from all the running processes, further assisting an analyst in reproducing the crash in case the automated Syzkaller fails to do so.

Future work Our work only exposes the impact of single-field corruptions, in reality an attacker usually has memory corruption capability of more than a single field, especially in use-after-frees when they can use the well known `msg_msg` [36] method to completely control all the data in the overlapping object. SCAVY also doesn't consider race condition bugs. An attacker may be able to exploit a corruption similar to the credential swapping method proposed by DirtyCred [23]. We leave such analysis as future work. It requires that the fuzzer introduces the corruption while a syscall is being executed. This imposes more challenges in terms of controlling the time window and reproducibility of the PoC. Another shortcoming of the way we search for sensitive fields is the detection of out-of-bound read privileges. Coined with the term elastic objects [4], a corruption in these structures allows the attacker to read out of bounds into kernel memory. This is undetectable by our fuzzer since the fuzzer doesn't rely on the corruption to issue an invalid system call with the expectation that it might work. Additionally, we mentioned throughout the paper that SCAVY may help existing bug exploration tools in reaching an exploitable state for more PoCs. We leave the implementation of a joins system as future work.

7 Conclusions

In this paper, we proposed a novel definition of exploitable state for Linux kernel exploits in the context of local unprivileged attacker. We also proposed a system to efficiently search for and detect such states at scale. Our system, SCAVY, introduces a novel runtime object tracking mechanism that avoids the disadvantages of static analysis based methods. Additionally, we develop a novel fuzzing method that achieves a balance between exploration breadth and depth such that it can better explore the depth of the weird state machine after memory corruptions in specific kernel structures while exploring corruptions in all possible fields of all kernel structures. We implemented a prototype of SCAVY and ran it on Linux 5.15.80, the long term supported version. Our analysis revealed that 17 fields in 11 kernel structures, when corrupted can lead the system into an exploitable state. At last, we discuss the implications of our findings from a defender's perspective believing that SCAVY will speed up the work on defending sensitive kernel fields.

References

- [1] AVLLAZAGAJ, E., ZHU, Z., BILGE, L., BALZAROTTI, D., AND DUMITRAŞ, T. When malware changed its mind: An empirical study of variable program behaviors in the real world. In *30th USENIX Security Symposium (USENIX Security 21)* (2021), pp. 3487–3504.
- [2] BRAND, M. In-the-Wild Series: Android Exploits, 2021. <https://googleprojectzero.blogspot.com/2021/01/in-wild-series-android-exploits.html>.
- [3] CHEN, W., ZOU, X., LI, G., AND QIAN, Z. {KOOBE}: Towards facilitating exploit generation of kernel {Out-Of-Bounds} write vulnerabilities. In *29th USENIX Security Symposium (USENIX Security 20)* (2020), pp. 1093–1110.
- [4] CHEN, Y., LIN, Z., AND XING, X. A systematic study of elastic objects in kernel exploitation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2020), CCS '20, Association for Computing Machinery, p. 1165–1184.
- [5] CHEN, Y., AND XING, X. Slake: Facilitating slab manipulation for exploiting vulnerabilities in the linux kernel. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (2019), pp. 1707–1722.
- [6] COOK, K. security things in linux v4.14, Nov 2017. <https://outflux.net/blog/archives/2017/11/14/security-things-in-linux-v4-14/>.
- [7] CORBET, J. Supervisor mode access prevention, Sep 2012. <https://lwn.net/Articles/517475/>.
- [8] CORBET, J. The slab and protected-memory allocators, 2018. <https://lwn.net/Articles/753154/>.
- [9] DAVANIAN, A., QI, Z., QU, Y., AND YIN, H. {DECAF++}: Elastic {Whole-System} dynamic taint analysis. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)* (2019), pp. 31–45.
- [10] FSECURELABS. KernelFuzzer: Cross Platform Kernel Fuzzer Framework, 2023. <https://github.com/FSecureLABS/KernelFuzzer>.

- [11] GARNIER, T. mm: Slab freelist randomization, Apr 2016. <https://lwn.net/Articles/685047/>.
- [12] GE, X., TALELE, N., PAYER, M., AND JAEGER, T. Fine-grained control-flow integrity for kernel software. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)* (2016), IEEE, pp. 179–194.
- [13] GMBH, B. F. S. Meltdown reloaded: Breaking windows kaslr by leaking kva shadow mappings, Jun 2020. <https://labs.bluefrostsecurity.de/blog/2020/06/30/meltdown-reloaded-breaking-windows-kaslr/>.
- [14] GOOGLE. Linux Kernel Sanitizers, fast bug-detectors for the Linux kernel, 2023. <https://github.com/google/kernel-sanitizers>.
- [15] H., L. Linux Kernel Heap Tampering Detection, 2009. <http://phrack.org/issues/66/15.html>.
- [16] HAN, H., AND CHA, S. K. Imf: Inferred model-based fuzzer. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2017), CCS '17, Association for Computing Machinery, p. 2345–2358.
- [17] JANG, Y., LEE, S., AND KIM, T. Breaking kernel address space layout randomization with intel tsx. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (2016), pp. 380–392.
- [18] JONES, D. Trinity: Linux system call fuzzer, 2023. <https://github.com/kernelslacker/trinity>.
- [19] KEMERLIS, V. P., POLYCHRONAKIS, M., AND KEROMYTIS, A. D. ret2dir: Rethinking kernel isolation. In *23rd USENIX Security Symposium (USENIX Security 14)* (2014), pp. 957–972.
- [20] KIM, K., JEONG, D. R., KIM, C. H., JANG, Y., SHIN, I., AND LEE, B. Hfl: Hybrid fuzzing on the linux kernel. In *NDSS* (2020).
- [21] LE, D. Linux Kernel Exploitation Technique: Overwriting modprobe_path, 2023. <https://lkmidas.github.io/posts/20210223-linux-kernel-pwn-modprobe/>.
- [22] LIN, Z., CHEN, Y., WU, Y., MU, D., YU, C., XING, X., AND LI, K. Grebe: Unveiling exploitation potential for linux kernel bugs. In *2022 IEEE Symposium on Security and Privacy (SP)* (2022), pp. 2078–2095.
- [23] LIN, Z., WU, Y., AND XING, X. Dirtycred: Escalating privilege in linux kernel. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (2022), pp. 1963–1976.
- [24] LINUX. Linux kernel coding style, 2022. <https://www.kernel.org/doc/html/v6.1/process/coding-style.html#allocating-memory>.
- [25] NIKOLENKO, V. Linux Kernel universal heap spray, 2018. <https://duasynt.com/blog/linux-kernel-heap-spray>.
- [26] NIKOLENKO, V. Practical SMEP bypass techniques on Linux, 2023. <http://bit.ly/3GVjvEn>.
- [27] NIKOLENKO, V. UAO mitigation for addr_limit overwrite, 2023. <https://duasynt.com/blog/android-uao-kernel-expl-mitigation>.
- [28] NUGROHO, Y. Exploiting the Futex Bug and uncovering Towelroot, 2014. <https://tinyhack.com/2014/07/07/exploiting-the-futex-bug-and-uncovering-towelroot/>.
- [29] POLOP, C. PEASS - Privilege Escalation Awesome Scripts SUITE, 2023. <https://github.com/carlospolop/PEASS-ng>.
- [30] POPOV, A. Four Bytes of Power: Exploiting CVE-2021-26708 in the Linux kernel, 2021. <https://a13xp0p0v.github.io/2021/02/09/CVE-2021-26708.html>.
- [31] ROSENBERG, D. Interesting kernel exploit posted, 2010. <https://lwn.net/Articles/419141/>.
- [32] RUTLAND, M., AND MARINAS, C. arm64: uaccess: remove set_fs(), 2020. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=3d2403fd10a1dbb359b154af41ffed9f2a7520e8>.
- [33] STONE, M. CVE-2019-2215: Android use-after-free in Binder, 2020. <https://googleprojectzero.blogspot.com/2019/11/bad-binder-android-in-wild-exploit.html>.
- [34] VYUKOV, D. Syzkaller, 2023. <https://github.com/google/syzkaller>.
- [35] WANG, D., ZHANG, Z., ZHANG, H., QIAN, Z., KRISHNAMURTHY, S. V., AND ABU-GHAZALEH, N. {SyzVegas}: Beating kernel fuzzing odds with reinforcement learning. In *30th USENIX Security Symposium (USENIX Security 21)* (2021), pp. 2741–2758.
- [36] WILL, 2021. <https://www.willsroot.io/2021/08/corctf-2021-fire-of-salvation-writeup.html>.
- [37] WU, W., CHEN, Y., XING, X., AND ZOU, W. {KEPLER}: Facilitating control-flow hijacking primitive evaluation for linux kernel vulnerabilities. In *28th USENIX Security Symposium (USENIX Security 19)* (2019), pp. 1187–1204.
- [38] WU, W., CHEN, Y., XU, J., XING, X., GONG, X., AND ZOU, W. {FUZE}: Towards facilitating exploit generation for kernel {Use-After-Free} vulnerabilities. In *27th USENIX Security Symposium (USENIX Security 18)* (2018), pp. 781–797.
- [39] YAMAUCHI, T., AKAO, Y., YOSHITANI, R., NAKAMURA, Y., AND HASHIMOTO, M. Additional kernel observer: Privilege escalation attack prevention mechanism focusing on system call privilege changes. *Int. J. Inf. Secur.* 20, 4 (aug 2021), 461–473.
- [40] YIN, H., AND SONG, D. Temu: Binary code analysis via whole-system layered annotative execution. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2010-3* (2010).
- [41] YIN, H., SONG, D., EGELE, M., KRUEGEL, C., AND KIRDA, E. Panorama: Capturing system-wide information flow for malware detection and analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security* (New York, NY, USA, 2007), CCS '07, Association for Computing Machinery, p. 116–127.
- [42] ZENG, K., CHEN, Y., CHO, H., XING, X., DOUPÉ, A., SHOSHITAISHVILI, Y., AND BAO, T. Playing for K(H)eaps: Understanding and improving linux kernel exploit reliability. In *31st USENIX Security Symposium (USENIX Security 22)* (Boston, MA, Aug. 2022), USENIX Association, pp. 71–88.
- [43] ZHAO, B., LI, Z., QIN, S., MA, Z., YUAN, M., ZHU, W., TIAN, Z., AND ZHANG, C. {StateFuzz}: System {Call-Based}{State-Aware} linux driver fuzzing. In *31st USENIX Security Symposium (USENIX Security 22)* (2022), pp. 3273–3289.
- [44] ZOU, X., AND QIAN, Z. CVE-2022-27666: Exploit esp6 modules in Linux kernel, 2022. <https://eternal.me/archives/1825>.